

บทที่ 11 : เว็บแอปพลิเคชันกับกฎหมาย และความมั่นคงปลอดภัย

1. การพัฒนาเว็บแอปพลิเคชันให้สอดคล้องกับความมั่นคงปลอดภัย
2. การพัฒนาเว็บแอปพลิเคชันให้สอดคล้องกับกฎหมาย
3. การติดตั้งเว็บแอปพลิเคชันด้วย Container Technology
4. ทดสอบความมั่นคงปลอดภัย



1. การพัฒนาเว็บแอปพลิเคชันให้สอดคล้องกับความมั่นคงปลอดภัย

- 1.1 การพัฒนา Register API ให้ปลอดภัย
- 1.2 การพัฒนา Login API ให้ปลอดภัย
- 1.3 การพัฒนา User Profile API ให้ปลอดภัย

1.1 การพัฒนา Register API ให้ปลอดภัย

- Insecure Register API

```
app.post('/v1/register', async (req: Request, res: Response) =>{  
  const { username, password, firstName, lastName } = req.body;  
  const sql = `INSERT INTO customer (username, password,  
    firstName, lastName) VALUES (?, ?, ?, ?)`;  
  
  await query(sql, [username, password, firstName, lastName]);  
  res.send({ message: 'Registration successful', status: true });  
});
```

• Secure Register API

```
app.post('/v2/register', [
  body('username').isString().notEmpty().withMessage('Username is required'),
  body('password').isLength({ min: 4 }).withMessage('Password must be at least 8 characters'),
  body('firstName').isString().notEmpty().withMessage('First name is required'),
  body('lastName').isString().notEmpty().withMessage('Last name is required')
], async (req: Request, res: Response, next: NextFunction) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).send({ message: 'Validation errors', errors: errors.array(), status: false });
  }

  const { username, password, firstName, lastName } = req.body;
  try {
    const existingUser = await query("SELECT * FROM customer WHERE username=?", [username]);
    if (existingUser.length > 0) {
      return res.status(409).send({ message: 'Username already exists', status: false });
    }

    const salt = await bcrypt.genSalt(10);
    const password_hash = await bcrypt.hash(password, salt);

    await query('INSERT INTO customer (username, password, firstName, lastName) VALUES (?, ?, ?, ?)',
      [username, password_hash, firstName, lastName]);

    res.send({ message: 'Registration successful', status: true });
  } catch (err) {
    next(err);
  }
});
```

ตารางที่ 1-1 : เปรียบเทียบ Insecure และ Secure Register API ตามแนวทางของ OWASP Top 10 : 2021

ช่องโหว่	คำอธิบาย	Register V1	Register V2	หมายเหตุ
A01: Broken Access Control	การควบคุมสิทธิ์ไม่ถูกต้อง	✅ ผ่าน (เพราะตั้งใจเปิดให้สมัครทั่วไปได้)	✅ ผ่าน (เพราะตั้งใจเปิดให้สมัครทั่วไปได้)	การเปิดให้ผู้ใช้ทั่วไป register ไม่ถือเป็นช่องโหว่
A02: Cryptographic Failures	การเข้ารหัสหรือการจัดการข้อมูลลับไม่ปลอดภัย	❌ บันทึกรหัสผ่านเป็น plaintext	✅ ใช้ bcrypt แฮชรหัสผ่านก่อนบันทึก	แก้ไขโดยใช้ salt + hash เพื่อป้องกันข้อมูลรั่ว
A03: Injection	SQL Injection	✅ ใช้ parameterized query แล้ว (ปลอดภัยระดับหนึ่ง)	✅ ใช้ parameterized query แล้ว (ปลอดภัยระดับหนึ่ง)	ทั้งสองเวอร์ชันใช้ ? ใน query ปลอดภัยจาก SQL injection อยู่แล้ว
A05: Security Misconfiguration	การตั้งค่าระบบไม่ปลอดภัย	❌ ไม่มี error handling	✅ ใช้ try/catch และ next(err)	ป้องกันการแสดง error stack ต่อผู้ใช้

ช่องโหว่	คำอธิบาย	Register V1	Register V2	หมายเหตุ
A07: Identification and Authentication Failures	ความผิดพลาดในระบบยืนยันตัวตน	✗ ไม่มีระบบจัดการรหัสผ่านปลอดภัย	✓ ใช้ bcrypt และตรวจสอบข้อมูลก่อน insert	แก้ไขให้ปลอดภัยขึ้นในการเก็บข้อมูลเข้าสู่ระบบ
A08: Software and Data Integrity Failures	ไม่มีการตรวจสอบความถูกต้องของข้อมูล	✗ ไม่มีการ validate input จาก client	✓ ใช้ express-validator ตรวจสอบความถูกต้องของข้อมูล	ป้องกันข้อมูลอันตรายหรือไม่ครบถ้วนเข้าสู่ระบบ
A09: Security Logging and Monitoring Failures	การขาดระบบบันทึกเหตุการณ์	✗ ไม่มีการ log หรือแจ้งเตือน	✗ ไม่มีการ log หรือแจ้งเตือน	ทั้งสองเวอร์ชันยังไม่มีระบบ logging หรือ monitoring

1.2 การพัฒนา Login API ให้ปลอดภัย

- Insecure Login API

```
app.post('/v1/login', async (req: Request, res: Response) => {  
  const {username, password} = req.body  
  let sql = "SELECT * FROM customer WHERE "  
  sql += "username='" + username + "'";  
  sql += " AND password = '" + password + "'";  
  console.log(sql);  
  // ' OR '1=1  
  const customer = await query(sql, [username, password]);  
  if (customer.length === 0) {  
    res.send({ message: 'Invalid username or password', status: false });  
  } else {  
    res.send({ message: 'Login successful', status: true } )  
  }  
});
```

• Secure Login API

```
app.post('/v2/login', [
  body('username').isString().notEmpty().withMessage('Username is required'),
  body('password').isString().notEmpty().withMessage('Password is required')
], async (req: Request, res: Response, next: NextFunction) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).send(
      { message: 'Validation errors',
        errors: errors.array(), status: false
      });
  }

  const { username, password } = req.body;
  try {
    const customer = await query("SELECT * FROM customer WHERE username=? AND isActive = 1", [username]);
    if (customer.length === 0) {
      return res.status(401).send({ message: 'Invalid username or password', status: false });
    }

    const isPasswordValid = bcrypt.compareSync(password, customer[0].password);
    if (!isPasswordValid) {
      return res.status(401).send({ message: 'Invalid username or password', status: false });
    }

    const token = jwt.sign(
      { custID: customer[0].custID, username: customer[0].username },
      SECRET_KEY, { expiresIn: '1h' });
    res.send(
      { custID: customer[0].custID, username, email: customer[0].email, token, message: 'Login successful', status: true }
    );
  } catch (err) {
    next(err);
  }
});
```

ตารางที่ 1-2 : เปรียบเทียบ Insecure และ Secure Login API ตามแนวทางของ OWASP Top 10 : 2021

ช่องโหว่	คำอธิบาย	Login V1	Login V2	หมายเหตุ
A01: Broken Access Control	การควบคุมสิทธิ์ไม่ถูกต้อง	✗ ไม่มีการจัดการสิทธิ์หรือการยืนยันตัวตนหลังล็อกอิน	✓ ใช้ JWT token เพื่อระบุตัวตนและควบคุมสิทธิ์	เพิ่มการตรวจสอบสิทธิ์ผู้ใช้ก่อนเข้าถึงข้อมูลภายใน
A02: Cryptographic Failures	การจัดการข้อมูลลับไม่ปลอดภัย	✗ ตรวจสอบรหัสผ่านที่เป็น plaintext โดยตรง	✓ ใช้ bcrypt แฮชรหัสผ่านก่อนตรวจสอบ	ป้องกันการรั่วไหลของรหัสผ่านเมื่อฐานข้อมูลถูกเจาะ
A03: Injection	การโจมตีแบบ SQL Injection	✗ ใช้ string concatenation ในคำสั่ง SQL	✓ ใช้ parameterized query ป้องกันการแทรกคำสั่ง	ลดความเสี่ยงจากคำสั่ง SQL ที่ผู้โจมตีแทรกเข้ามา
A05: Security Misconfiguration	การตั้งค่าความปลอดภัยผิดพลาด	✗ ไม่มีระบบจัดการ error อย่างเหมาะสม	✓ ใช้ next(err) เพื่อส่ง error ไปยัง middleware	ลดโอกาสเปิดเผยข้อมูลภายในระบบ

ช่องโหว่	คำอธิบาย	Login V1	Login V2	หมายเหตุ
A07: Identification and Authentication Failures	การระบุตัวตนและการยืนยันตัวตนล้มเหลว	✗ ไม่มีการตรวจสอบรูปแบบ input หรือ token	✓ ตรวจสอบ input ด้วย express-validator, ใช้ bcrypt และ JWT	ทำให้การยืนยันตัวตนถูกต้องและปลอดภัยตามมาตรฐาน
A09: Security Logging and Monitoring Failures	การบันทึกและตรวจสอบเหตุการณ์ความปลอดภัย	✗ ไม่มีการบันทึกความพยายามล็อกอินล้มเหลว	⚠ มีการจัดการ error แต่ยังขาดระบบ logging เต็มรูปแบบ	ยังควรเพิ่มระบบบันทึกการพยายามล็อกอินเพื่อเฝ้าระวังการโจมตี

1.3 การพัฒนา User Profile API ให้ปลอดภัย

- Insecure User Profile API

```
app.get('/v1/profile/:id', async (req: Request, res: Response) => {  
  const custID = req.params.id;  
  let sql = "SELECT * FROM customer WHERE "  
  | sql += "custID=" + custID;  
  | sql += " AND isActive = 1";  
  //1--  
  const customer = await query(sql);  
  if (customer.length === 0) {  
    | res.send({ message: 'Customer not found', status: false });  
  }else{  
    | res.send({ ...customer[0], message: 'Success', status: true });  
    | }  
  });
```

- Secure User Profile API

```
app.get('/v2/profile/:id', validateToken,
  async (req: Request, res: Response, next: NextFunction) => {
    const custID = req.params.id;
    const user = (req as any).user as Jwpayload;

    if (Number(custID) !== user.custID) {
      return res.status(403).send({ message: 'Access denied', status: false });
    }

    try {
      const customer = await query(`
        SELECT custID, username, firstName, lastName, address, mobilePhone, gender, email, imageFile,
        IF(birthdate IS NOT NULL, DATE_FORMAT(birthdate, '%Y-%m-%d'), NULL) AS birthdate
        FROM customer
        WHERE custID = ? AND isActive = 1`, [custID]);

      if (customer.length === 0) {
        return res.status(404).send({ message: 'Customer not found', status: false });
      }
      res.send({ ...customer[0], message: 'Success', status: true });
    } catch (err) {
      next(err);
    }
  });
```

ตารางที่ 1-3 : เปรียบเทียบ Insecure และ Secure User Profile API ตามแนวทางของ OWASP Top 10 : 2021

ช่องโหว่	คำอธิบาย	User Profile V1	User Profile V2	หมายเหตุ
A01: Broken Access Control	การควบคุมสิทธิ์ไม่ถูกต้อง	✗ ไม่มีการตรวจสอบสิทธิ์ ผู้ใช้สามารถเข้าถึงข้อมูลลูกค้าคนอื่นได้โดยเปลี่ยน custID	✓ ใช้ validateToken และตรวจสอบว่า custID จาก token ต้องตรงกับ custID ที่ร้องขอ	ป้องกันผู้ใช้เข้าถึงข้อมูลของผู้อื่น
A02: Cryptographic Failures	การจัดการข้อมูลลับไม่ปลอดภัย	✗ ไม่มีการเข้ารหัสการส่งข้อมูล (token หรือตรวจสอบสิทธิ์)	✓ ใช้ JWT token ที่มีการเซ็นดิจิทัลป้องกันการปลอมแปลง	เพิ่มความปลอดภัยในการสื่อสาร และป้องกันการเข้าถึงโดยไม่ได้รับอนุญาต
A03: Injection	การป้องกัน SQL Injection ไม่เหมาะสม	✗ ใช้การต่อสตริง SQL ("custID=" + custID) เสี่ยงถูกแทรกคำสั่ง	✓ ใช้ Parameterized Query (WHERE custID = ?)	ป้องกันการโจมตีแบบ SQL Injection ที่อาจดึงหรือเปลี่ยนข้อมูล
A05: Security Misconfiguration	การตั้งค่าความปลอดภัยไม่เหมาะสม	⚠ ไม่มีการจัดการ error หรือ response ที่เหมาะสม อาจเผยข้อมูลโครงสร้าง	✓ ใช้ try-catch และส่งข้อความแสดงผลแบบจำกัด	ป้องกันการเปิดเผยรายละเอียดของระบบใน error response

ช่องโหว่	คำอธิบาย	User Profile V1	User Profile V2	หมายเหตุ
A07: Identification and Authentication Failures	การระบุตัวตนหรือการตรวจสอบสิทธิ์ไม่ถูกต้อง	✗ ไม่มีการยืนยันตัวตนก่อนเข้าถึงโปรไฟล์	✓ ใช้ JWT token เพื่อระบุตัวตนผู้ใช้	เพิ่มความถูกต้องของการระบุตัวตนและป้องกันการ spoof identity
A08: Software and Data Integrity Failures	ความสมบูรณ์ของซอฟต์แวร์และข้อมูลไม่ถูกตรวจสอบ	✗ ไม่มีการตรวจสอบความถูกต้องของข้อมูล token	✓ JWT มี signature ป้องกันการปลอมแปลงข้อมูล	ป้องกันการแก้ไขข้อมูลระหว่างส่ง
A09: Security Logging and Monitoring Failures	ไม่มีระบบ log และตรวจสอบเหตุการณ์	✗ ไม่มีการบันทึก log การเข้าถึง profile	✓ สามารถเพิ่ม middleware logging / audit ภายหลังจาก validateToken	ช่วยติดตามการเข้าถึงผิดปกติและการโจมตี

2. การพัฒนาเว็บแอปพลิเคชันให้สอดคล้องกับกฎหมาย

- **หน้าสมัครสมาชิก**

- **PDPA มาตรา 19** : การขอความยินยอม (Consent)
- **PDPA มาตรา 21** : ประกาศการคุ้มครองข้อมูลส่วนบุคคล (Privacy Notice) / นโยบายคุ้มครองข้อมูลส่วนบุคคล (Privacy Policy)
- **PDPA มาตรา 22** : เก็บข้อมูลส่วนบุคคลเท่าที่จำเป็น
- **พ.ร.บ. คอมฯ มาตรา 11** : การส่งอีเมลโฆษณาที่สามารถกดยกเลิกการรับบริการได้

- **หน้าเข้าสู่ระบบ**

- **พ.ร.บ. คอมฯ มาตรา 26** : การเก็บข้อมูลจราจรคอมพิวเตอร์ (Traffic Logging)

- **หน้าแรก**

- **PDPA มาตรา 21** : ประกาศการคุ้มครองข้อมูลส่วนบุคคล (Privacy Notice)/นโยบายคุกกี้ (Cookie Policy)

- **หน้าโปรไฟล์ผู้ใช้**

- **PDPA มาตรา 30** : สิทธิการขอเข้าถึงข้อมูลส่วนบุคคล
- **PDPA มาตรา 33** : สิทธิในการลบหรือทำลาย
- **PDPA มาตรา 35** : สิทธิในการขอให้แก้ไขข้อมูลส่วนบุคคลให้ถูกต้อง

3. การติดตั้งเว็บแอปพลิเคชันด้วย Container Technology

- 3.1 การติดตั้ง Docker Desktop
- 3.2 การสร้างไฟล์ Dockerfile
- 3.3 การสร้างไฟล์ docker-compose.yml
- 3.4 การ Build Image และ Run Container
- 3.5 การ Push Image ขึ้น Docker Hub
- 3.6 การจำลอง Server
- 3.7 การ Pull Image และ Run Container บน Server

3.1 การติดตั้ง Docker Desktop

ขั้นตอนที่ 1 : ตรวจสอบและเปิดใช้งาน Hardware Virtualization

☰

Processes

Performance

App history

Startup apps

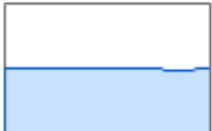
Users

Details

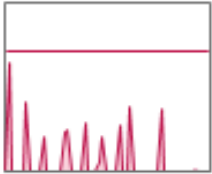
Services

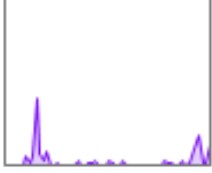
Performance

 **CPU**
20% 2.83 GHz

 **Memory**
8.4/15.9 GB (53%)

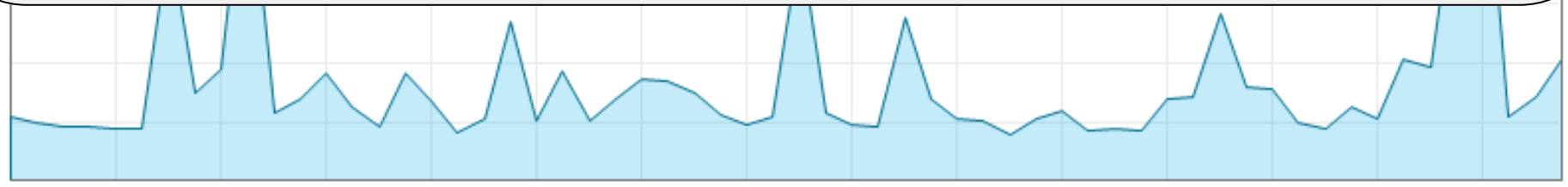
 **Disk 0 (G: C: D:)**
SSD (NVMe)
4%

 **Wi-Fi**
Wi-Fi
S: 32.0 R: 8.0 Kbps

 **GPU 0**
Intel(R) UHD Graphics ...
13%

- **เปิด Task Manager:** กดปุ่ม Ctrl + Shift + Esc พร้อมกัน หรือคลิกขวาที่แถบ Taskbar แล้วเลือก Task Manager
- **ไปที่แท็บ Performance:** คลิกที่แท็บ Performance ที่อยู่ด้านซ้ายมือ
- **เลือก CPU:** คลิกที่ CPU
- **ตรวจสอบ Virtualization:** ที่ด้านล่างของหน้าจอ ส่วนของ CPU หากคุณเห็น Virtualization: Enabled แสดงว่าเปิดใช้งานแล้ว หากเป็น Disabled แสดงว่าปิดใช้งานอยู่ **ให้ทำการเปิดใช้งาน Hardware Virtualization (Intel VT-x / AMD-V) ที่ BIOS**

60 seconds



0

Utilization	Speed	Base speed:	1.80 GHz
20%	2.83 GHz	Sockets:	1
Processes	Threads	Cores:	4
254	3257	Logical processors:	8
Handles	124312	Virtualization:	Enabled
...	...	L1 cache:	256 KB

ขั้นตอนที่ 2 : ตรวจสอบและเปิดใช้งาน WSL 2

- เปิดใช้งาน wsl ด้วยคำสั่ง ดังนี้

```
C:\> wsl --install
```

เมื่อเปิดใช้งาน wsl ตัว ubuntu จะถูกติดตั้งมาให้อัตโนมัติ หากต้องการติดตั้ง distro อื่น เช่น kali linux ก็สามารถทำได้

- ตรวจสอบเวอร์ชัน ดังนี้

```
C:\> wsl --version
```

- ตรวจสอบ distro ที่ได้ทำการติดตั้งแล้ว ดังนี้

```
C:\> wsl --list
```

- หากต้องการตรวจสอบว่ามี distro อะไรบ้างที่สามารถติดตั้งได้ ให้ใช้คำสั่ง ดังนี้

```
C:\> wsl --list --online
```

- หากต้องการติดตั้ง wsl พร้อม kali linux ให้ใช้คำสั่ง ดังนี้

```
C:\> wsl --install -d kali-linux
```

ในติดตั้งระบบจะให้ทำการกำหนด username และ password สำหรับ linux

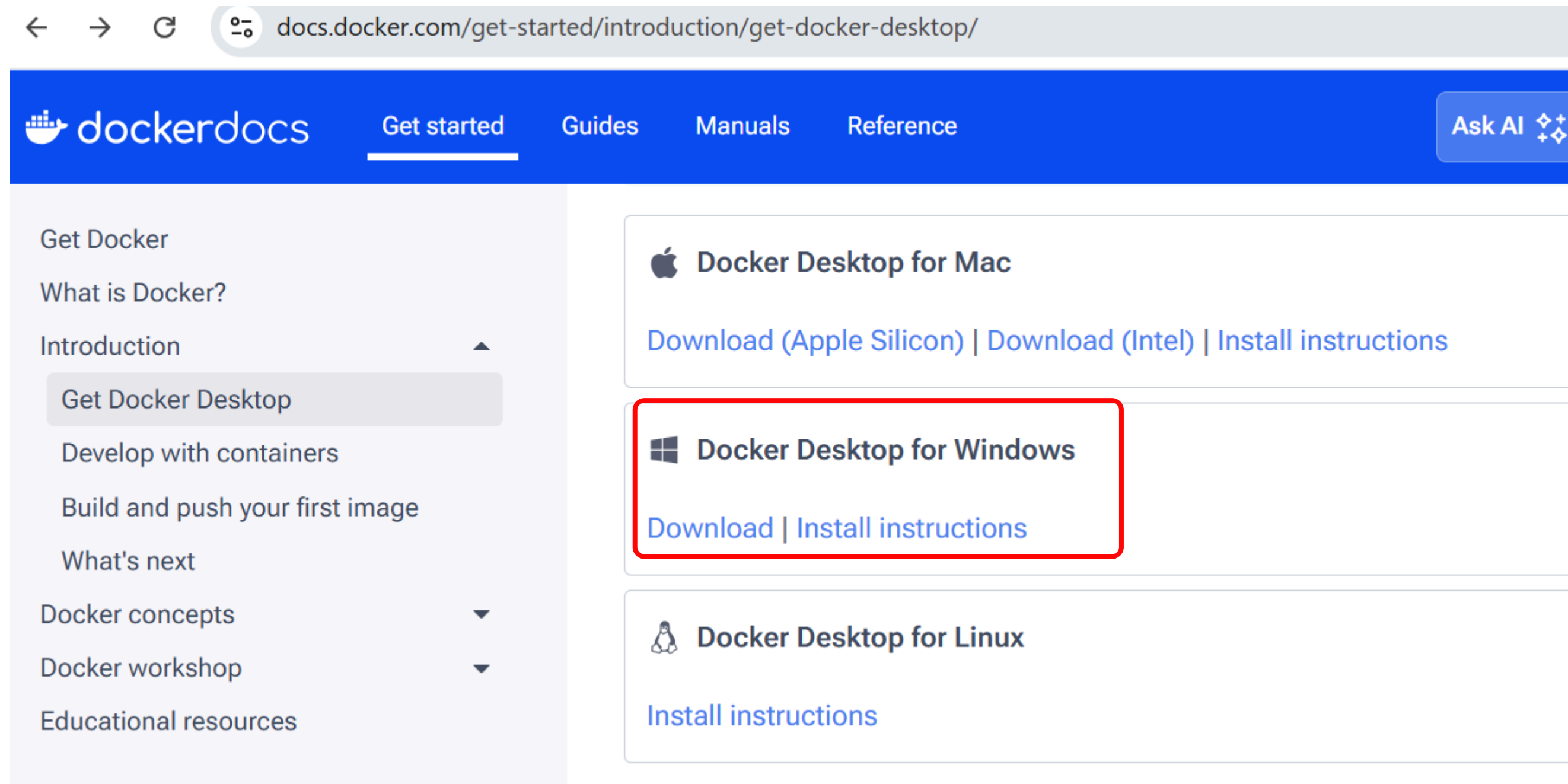
- หากต้องการอัปเดตเวอร์ชัน ให้ใช้คำสั่ง ดังนี้

```
C:\> wsl --update
```

ขั้นตอนที่ 3 : ดาวน์โหลดและติดตั้ง Docker Desktop

- ดาวน์โหลด Docker Desktop จากเว็บไซต์

<https://docs.docker.com/get-started/introduction/get-docker-desktop/>

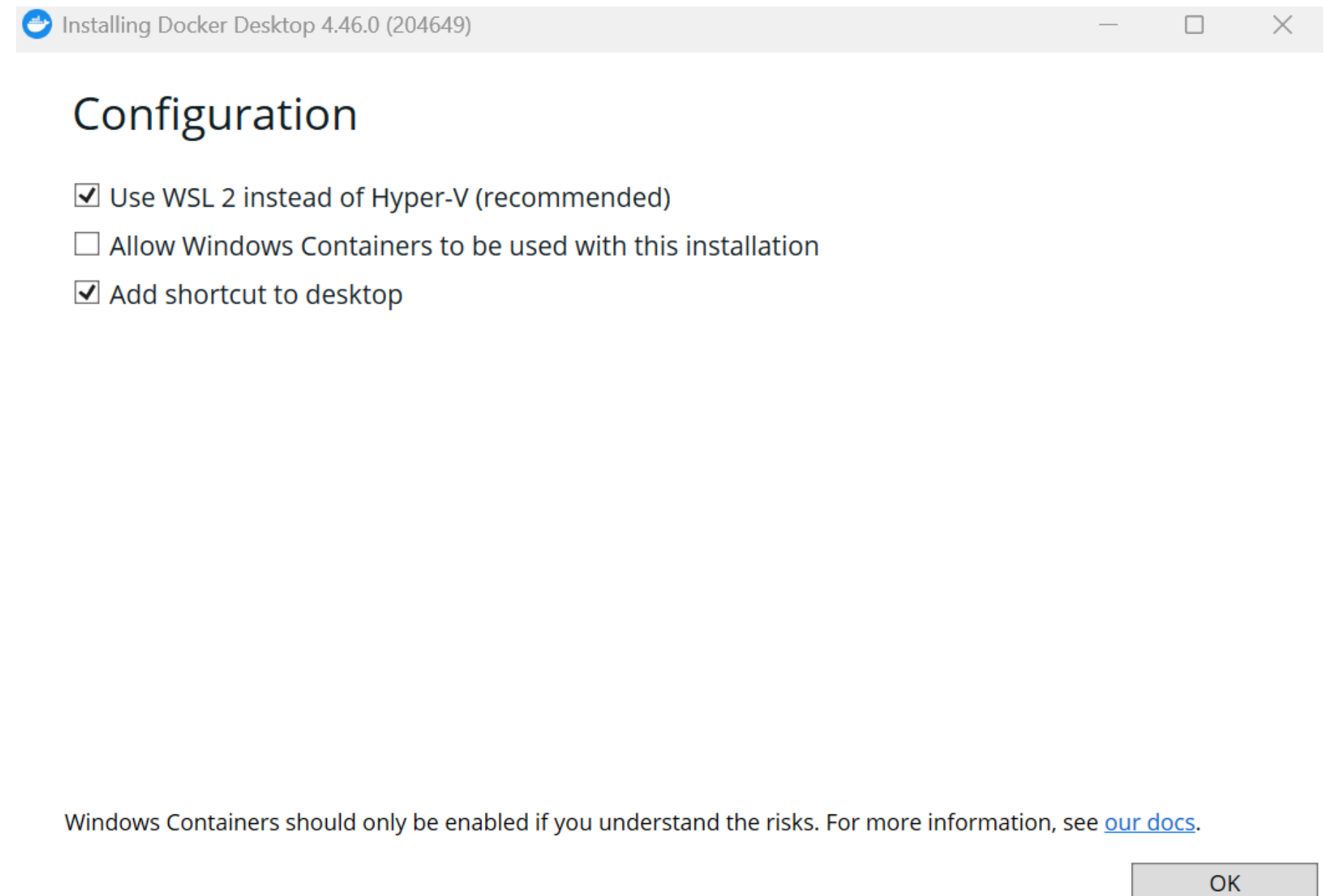


- ติดตั้ง Docker Desktop

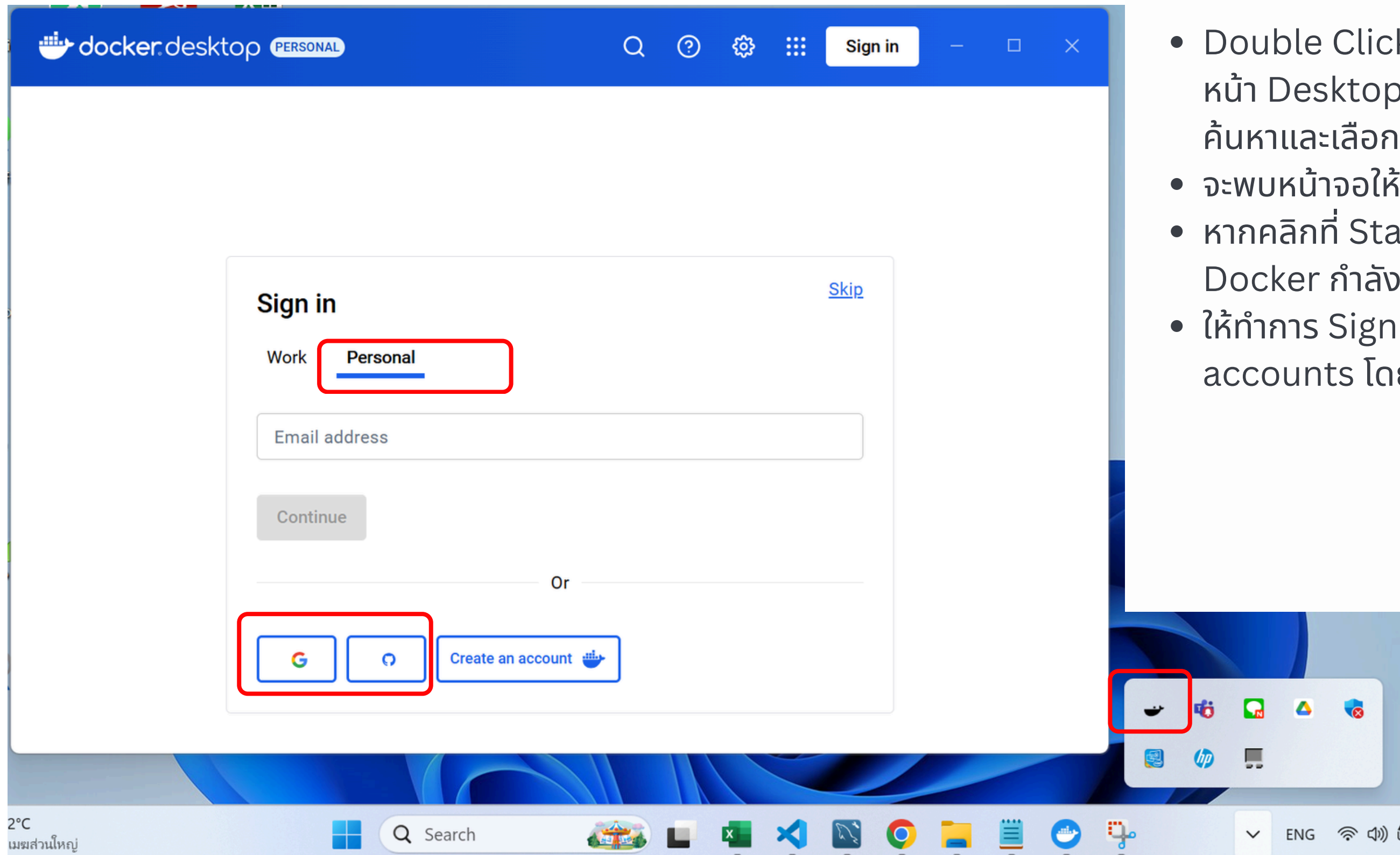
- รับไฟล์ติดตั้งที่ดาวน์โหลดมา (Docker Desktop Installer.exe)
- หาก Windows แจ้งข้อสิทธิในการติดตั้ง ให้คลิกปุ่ม “yes”
- เลือก “Use WSL 2” (แนะนำ)
- เลือก “Add shortcut to desktop” (แนะนำ)
- กด OK / Next ไปเรื่อย ๆ จนติดตั้งเสร็จ

- ตรวจสอบเวอร์ชัน ดังนี้

```
C:\> docker --version
```

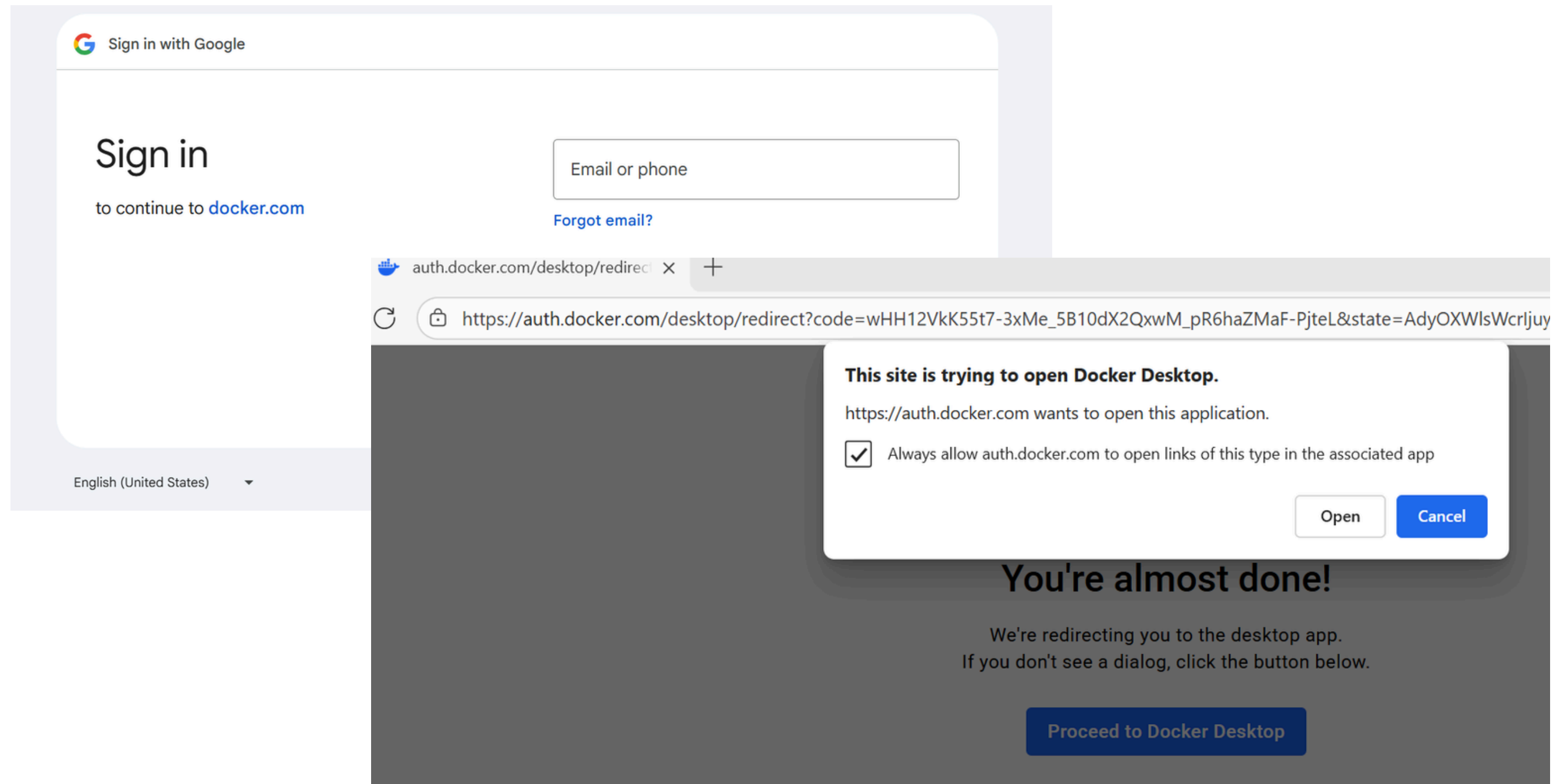


ขั้นตอนที่ 4 : เปิดใช้งาน Docker Desktop และสมัคร Docker Hub

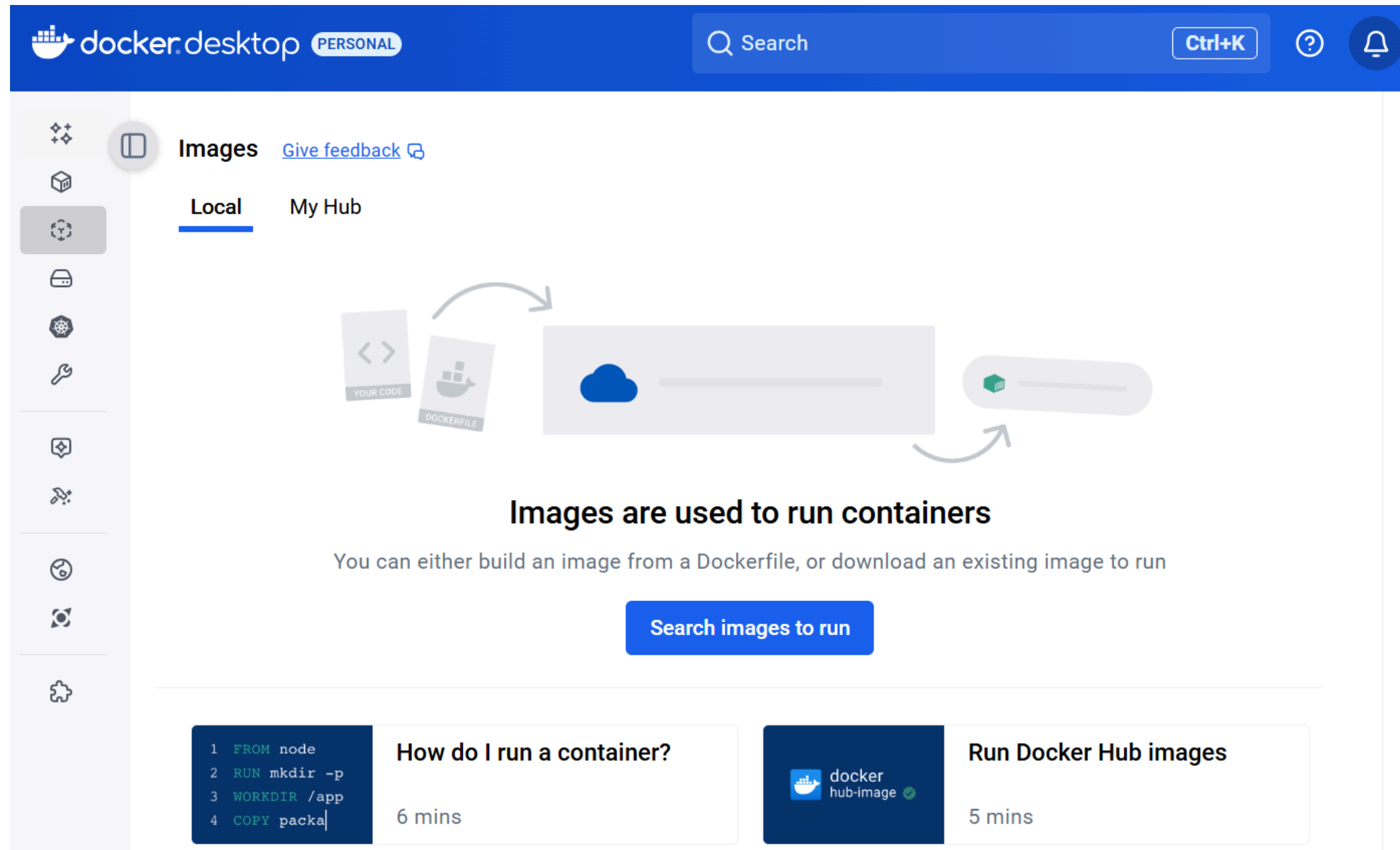


- Double Click ไอคอน Docker Desktop ที่ หน้า Desktop หรือ กดปุ่ม Windows แล้ว ค้นหาและเลือกคำว่า Docker Desktop
- จะพบหน้าจอให้ทำการ Sign in
- หากคลิกที่ Status Bar จะพบว่า มีไอคอนของ Docker กำลังทำงาน
- ให้ทำการ Sign in ด้วย Gmail หรือ GitHub accounts โดยการคลิกเลือกอย่างใดอย่างหนึ่ง

- ดำเนินการ Sign In ตามคำสั่งจนสำเร็จ



- ดูรายการ Docker Image ที่เก็บไว้บน**เครื่อง Computer** โดยการคลิกที่ Images>Local

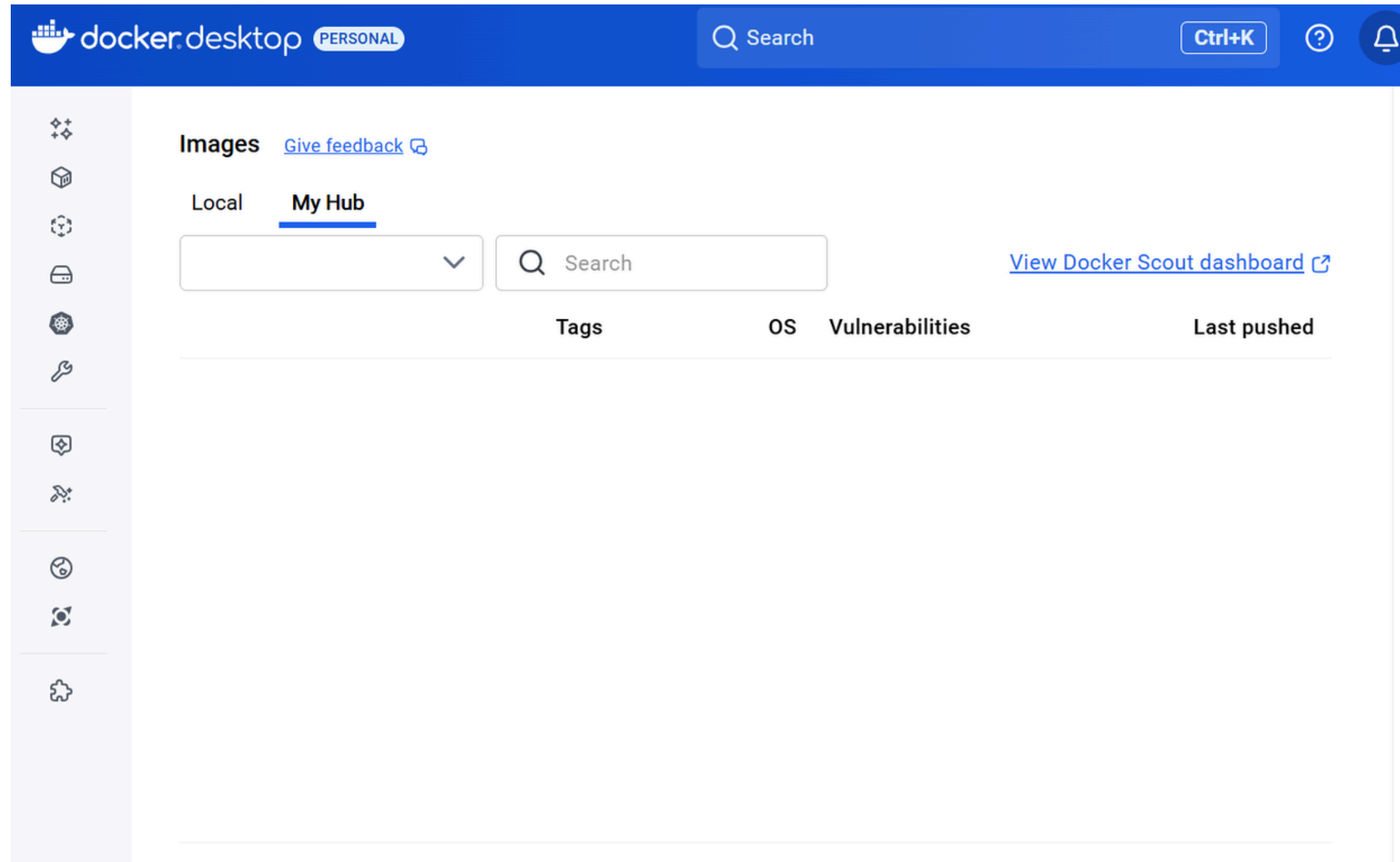


- จากตัวอย่างยังไม่มีรายการ Docker Image

- คำสั่งสำหรับตรวจสอบรายการ docker image

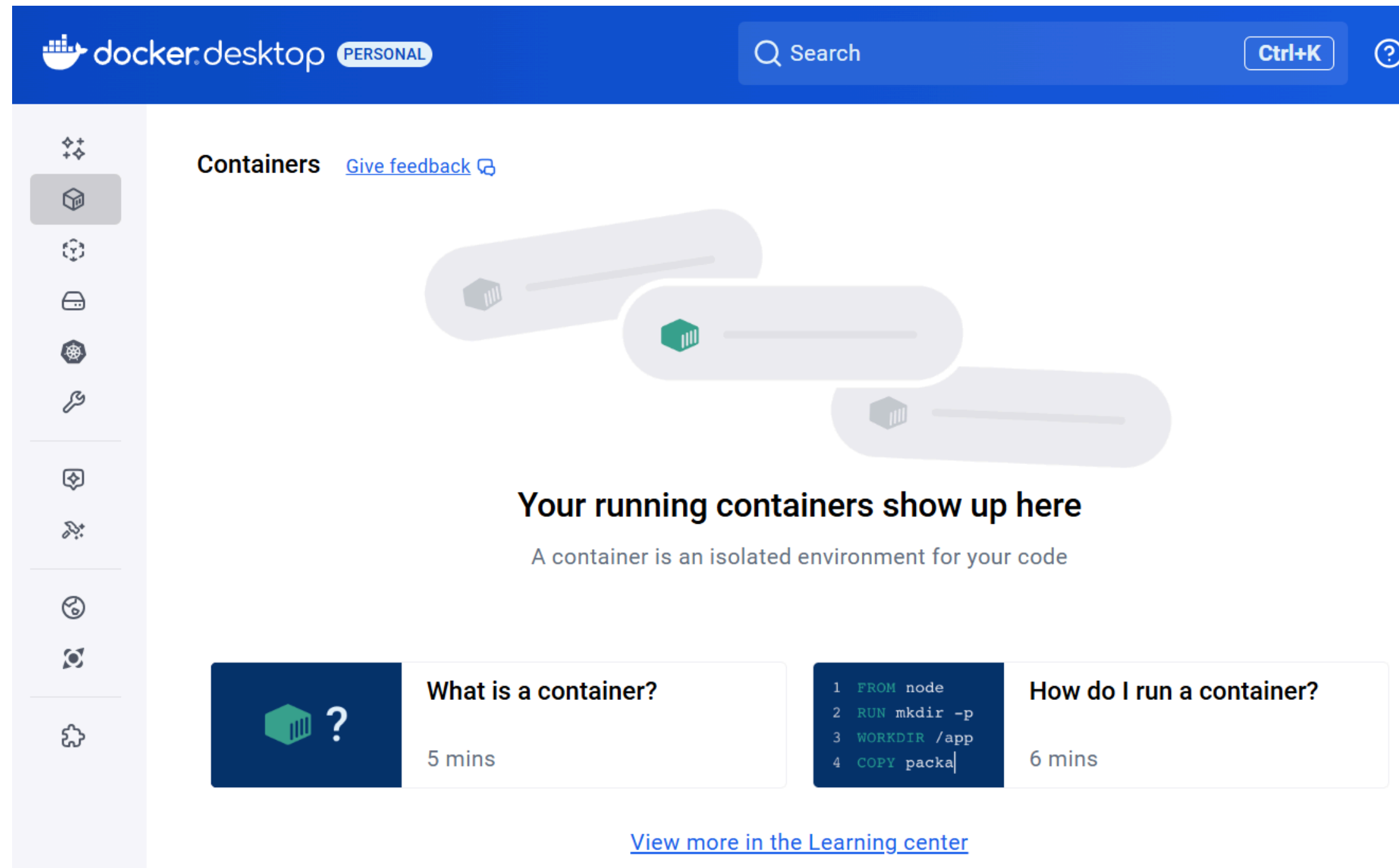
```
C:\> docker images
```

- ดูรายการ Docker Image ที่เก็บไว้บน **Docker Hub** โดยการคลิกที่ Images>My Hub



- เราสามารถอัปโหลด Docker Image ไปเก็บไว้บน Docker Hub คล้าย ๆ กับ GitHub

- ดูรายการ Docker Container โดยคลิกที่แถบ containers



- Docker Container คือ instance ที่เกิดจากการรัน Docker image
- จากตัวอย่างยังไม่มีรายการ Docker Container
- คำสั่งสำหรับตรวจสอบรายการ Docker Container

```
C:\> docker ps -a
```

3.2 การสร้างไฟล์ Dockerfile

- สร้าง Dockerfile สำหรับ Express.js API

C:\shopdee\backend\express\Dockerfile

```
1. # Dockerfile for Express.js API
2. FROM node:24-slim AS builder
3. WORKDIR /app
4. COPY package.json package-lock.json ./
5. RUN npm ci
6. COPY . .
7. RUN npx tsc
8.
9. FROM node:24-slim AS runner
10. WORKDIR /app
11.
12. # Install netcat
13. RUN apt-get update && \
14. apt-get install -y --no-install-recommends netcat-openbsd && \
15. rm -rf /var/lib/apt/lists/*
16.
17. # Copy only production dependencies
18. COPY package.json package-lock.json ./
19. RUN npm ci --omit=dev
20.
21. # Copy compiled code and assets from builder
22. COPY --from=builder /app/dist ./dist
23. COPY --from=builder /app/assets ./assets
24.
25. # Create wait-for script
26. RUN echo '#!/bin/sh' > /wait-for.sh && \
27. echo 'set -e' >> /wait-for.sh && \
28. echo 'host="$1"' >> /wait-for.sh && \
29. echo 'shift' >> /wait-for.sh && \
30. echo 'until nc -z "$host" 3306; do' >> /wait-for.sh && \
31. echo 'echo "Waiting for MySQL at $host:3306..." >> /wait-for.sh && \
32. echo 'sleep 2' >> /wait-for.sh && \
33. echo 'done' >> /wait-for.sh && \
34. echo 'echo "MySQL is up - starting application"' >> /wait-for.sh && \
35. echo 'exec "$@"' >> /wait-for.sh && \
36. chmod +x /wait-for.sh
37.
38. ENV NODE_ENV=production
39. EXPOSE 4000
40. ENTRYPOINT ["/wait-for.sh", "mysql"]
41. CMD ["node", "dist/index.js"]
```

แล้วแก้ไข Environment Variables ใน
C:\shopdee\backend\express\.env เช่น
DB_HOST=mysql หรือ service name ของ MySQL

- สร้าง Dockerfile สำหรับ Flask API (กรณีต้องการ Run ML Model)

C:\shopdee\backend\flask\Dockerfile

```
1. FROM python:3.12-slim
2. WORKDIR /app
3.
4. COPY ./requirements.txt ./requirements.txt
5. RUN pip install --no-cache-dir -r requirements.txt
6.
7. COPY ..
8.
9. EXPOSE 5000
10. CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "api:app"]
```

แล้วแก้ไข Environment Variables ใน
C:\shopdee\backend\flask\.env เช่น
DB_HOST=mysql หรือ service name ของ MySQL

- สร้าง Dockerfile สำหรับ Next.js

C:\shopdee\frontend\nextjs\Dockerfile

```
1.FROM node:24-slim AS deps
2.WORKDIR /app
3.COPY package.json package-lock.json ./
4.RUN npm ci
5.
6.FROM node:24-slim AS builder
7.WORKDIR /app
8.COPY --from=deps /app/node_modules ./node_modules
9.COPY . .
10.RUN npm run build
11.
12.FROM node:24-slim AS runner
13.WORKDIR /app
14.ENV NODE_ENV=production
15.COPY --from=builder /app/.next ./next
16.COPY --from=builder /app/public ./public
17.COPY --from=builder /app/package.json ./package.json
18.COPY --from=builder /app/node_modules ./node_modules
19.
20.EXPOSE 3000
21.CMD ["npm", "run", "start"]
```

แล้วแก้ไข Environment Variables ใน

C:\shopdee\frontend\nextjs\.env เช่น

NEXT_PUBLIC_API_BASE_URL=http://localhost/api/node

API_BASE_URL=http://express:4000/api

ML_BASE_URL=http://flask:5000/api

- สร้าง Dockerfile สำหรับ Flutter (กรณีต้องการ Run Mobile App)

C:\shopdee\frontend\flutter\Dockerfile

```
1. FROM ghcr.io/cirruslabs/flutter:stable AS build
2.
3. WORKDIR /app
4.
5. COPY ..
6.
7. RUN flutter config --enable-web
8. RUN flutter clean
9. RUN flutter pub get
10. #RUN touch .env
11.
12. RUN flutter build web
13.
14. FROM nginx:stable-alpine
15. COPY --from=build /app/build/web /usr/share/nginx/html
16. EXPOSE 80
17. CMD ["nginx", "-g", "daemon off;"]
```

แล้วแก้ไข Environment Variables ใน
C:\shopdee\frontend\flutter\.env เช่น
API_URL=http://localhost/api/node
ML_URL=http://localhost/api/flask

- สร้างไฟล์คอนฟิกของ Nginx

C:\shopdee\frontend\nginx\default.conf

```
1. server {
2.
3.     listen 80;
4.     server_name _;
5.
6.     #####
7.     # Next.js
8.     #####
9.     location / {
10.
11.         proxy_pass http://nextjs:3000;
12.         proxy_http_version 1.1;
13.         proxy_set_header Host $host;
14.         proxy_set_header X-Real-IP $remote_addr;
15.         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
16.         proxy_set_header X-Forwarded-Proto $scheme;
17.         proxy_set_header Upgrade $http_upgrade;
18.         proxy_set_header Connection "upgrade";
19.     }
20.
21.
22.     #####
23.     # Express API
24.     #####
25.     location /api/node/ {
26.         proxy_pass http://express:4000/api/;
27.         proxy_http_version 1.1;
28.         proxy_set_header Host $host;
29.         proxy_set_header X-Real-IP $remote_addr;
30.         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
31.         proxy_set_header X-Forwarded-Proto $scheme;
32.     }
33.
34.     #####
35.     # Flask API
36.     #####
37.     location /api/flask/ {
38.         proxy_pass http://flask:5000/api/;
39.         proxy_http_version 1.1;
40.         proxy_set_header Host $host;
41.         proxy_set_header X-Real-IP $remote_addr;
42.         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
43.         proxy_set_header X-Forwarded-Proto $scheme;
44.     }
45.
46. }
```

3.3 การสร้างไฟล์ docker-compose.yml

- สร้าง docker-compose.yml สำหรับ Backend
 - export ฐานข้อมูล MySQL แล้วเก็บไว้ที่ C:\shopdee\backend\mysql\schema.sql
 - สร้าง docker-compose.yml แล้วเก็บไว้ที่ C:\shopdee\backend\docker-compose.yml

```
1.services:
2.  mysql:
3.    image: mysql:9.7
4.    container_name: mysql
5.    restart: always
6.    expose:
7.      - "3306"
8.    environment:
9.      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD:-1234}
10.     MYSQL_DATABASE: ${DB_NAME:-shopdee}
11.  volumes:
12.    - mysql_data:/var/lib/mysql
13.    - ./mysql/schema.sql:/docker-entrypoint-initdb.d/schema.sql:ro
14.  healthcheck:
15.    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
16.    interval: 10s
17.    timeout: 5s
18.    retries: 5
19.
20.  express:
21.    build: ./express
22.    image: ${DOCKER_USER:-local}/express:latest
23.    container_name: express
24.    restart: always
25.    expose:
26.      - "4000"
27.    env_file:
28.      - express/.env
29.    depends_on:
30.      - mysql
31.
32.  flask:
33.    build: ./flask
34.    image: ${DOCKER_USER:-local}/flask:latest
35.    container_name: flask
36.    restart: always
37.    expose:
38.      - "5000"
39.    env_file:
40.      - flask/.env
41.    depends_on:
42.      - mysql
43.
44.  volumes:
45.    mysql_data:
```

หากต้องการ Push ไฟล์ Image ไปยัง Docker Hub
ในไฟล์ C:\shopdee\frontend\.env
ให้ระบุ docker username ตรงตัวแปร **DOCKER_USER**

- สร้าง docker-compose.yml สำหรับ Frontend
C:\shopdee\frontend\docker-compose.yml

```
1.services:
2. nextjs:
3. build: ./nextjs
4. image: ${DOCKER_USER:-local}/nextjs:latest
5. container_name: nextjs
6. restart: always
7. expose:
8. - "3000"
9. env_file:
10. - nextjs/.env
11. depends_on:
12. - express
13. - flask
14.
15. flutter:
16. build: ./flutter
17. image: ${DOCKER_USER:-local}/flutter:latest
18. container_name: flutter
19. restart: always
20. env_file:
21. - flutter/.env
22. depends_on:
23. - express
24. - flask
25.
26. nginx:
27. image: nginx:latest
28. container_name: nginx
29. restart: always
30. ports:
31. - "80:80"
32. volumes:
33. - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
34. depends_on:
35. - nextjs
36. - flutter
```

- สร้าง docker-compose.yml สำหรับรวม Backend กับ Frontend
C:\shopdee\docker-compose.yml

```
1.include:  
2. - ./backend/docker-compose.yml  
3. - ./frontend/docker-compose.yml  
4.  
5.networks:  
6. default:  
7.   external: true  
8.   name: shopdee_network
```

3.4 การ Build Image และ Run Container

- ทำการ build และ run container

```
C:\shopdee> docker compose up -d --build
```

- หากต้องการ build และ run container แยกทีละ service

```
C:\shopdee> docker compose up -d --build nextjs
```

- หากต้องการ build แยกทีละ service

```
C:\shopdee> docker compose build nextjs
```

- หากต้องการ run แยกทีละ service

```
C:\shopdee> docker compose up -d nextjs
```

3.5 การ Push Image ขึ้น Docker Hub

3.5.1 การ Push Image ขึ้น Docker Hub โดยใช้ GUI

The screenshot displays the Docker Desktop interface. The 'Images' tab is selected in the sidebar. The main panel shows a list of local images. A context menu is open over the 'local/nextjs' image, highlighting the 'Push to Docker Hub' option. The table below lists the images shown in the screenshot.

Name	Tag	Image ID	Created	Size
local/nextjs	latest	11ceb3e96847	2 days ago	
local/flutter	latest	1bd0a6fec4c9	2 days ago	153.22 MB
local/flask	latest	7029ece375b4	2 days ago	747.36 MB
local/express	latest	55a9630779c5	3 days ago	430.95 MB
nginx	latest	ec4ed8b5299e	7 days ago	240.55 MB
mysql	9.7	ad88e1c86cbf	7 days ago	1.28 GB

- เปิด Docker Desktop
- Login (กรณียังไม่ Login/Sign In)
- คลิกที่แถบเมนูด้านซ้าย “Images”
- คลิกที่จุด 3 จุด เพื่อเลือก Image File ที่ต้องการ แล้วจะมี Popup ขึ้นมา
- คลิกที่ “Push to Docker Hub”
- เข้าไปยังเว็บไซต์ Docker Hub เพื่อตรวจสอบ Image File ที่ Push ขึ้นไป

หมายเหตุ : ต้องเปลี่ยนชื่อ image ในส่วนหน้าของเครื่องหมาย / จาก **local** เป็น **username** ควรเปลี่ยนตั้งแต่ขั้นตอนของการ build images ตามที่อธิบายไว้ในหัวข้อ 3.3

3.5.2 การ Push Image ขึ้น Docker Hub โดยใช้ CLI

- login

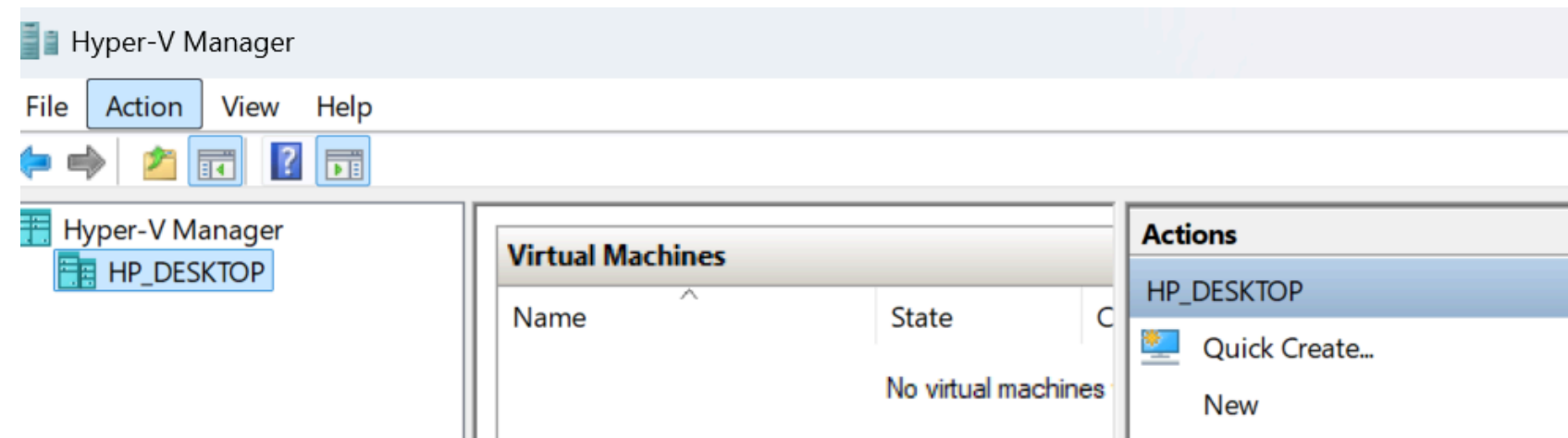
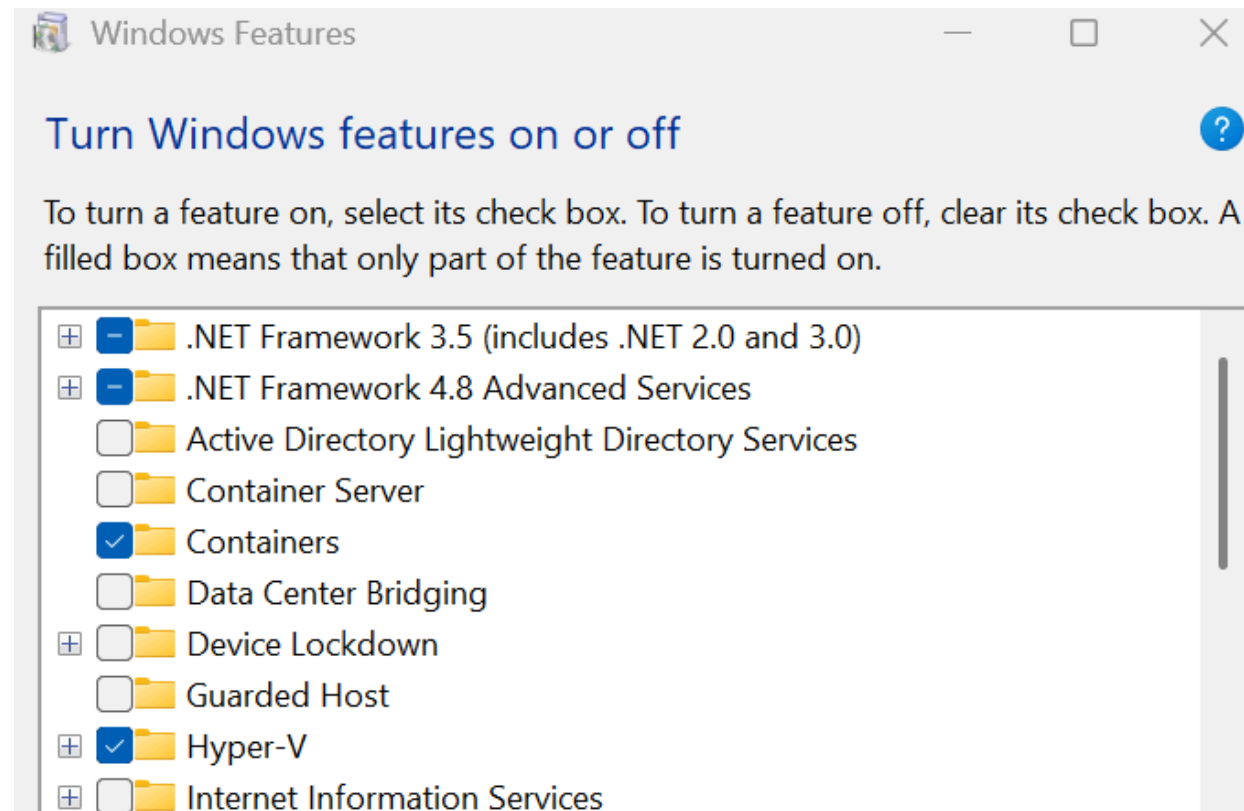
```
C:\shopdee> docker login -u username
```

- ทำการ push image เช่น

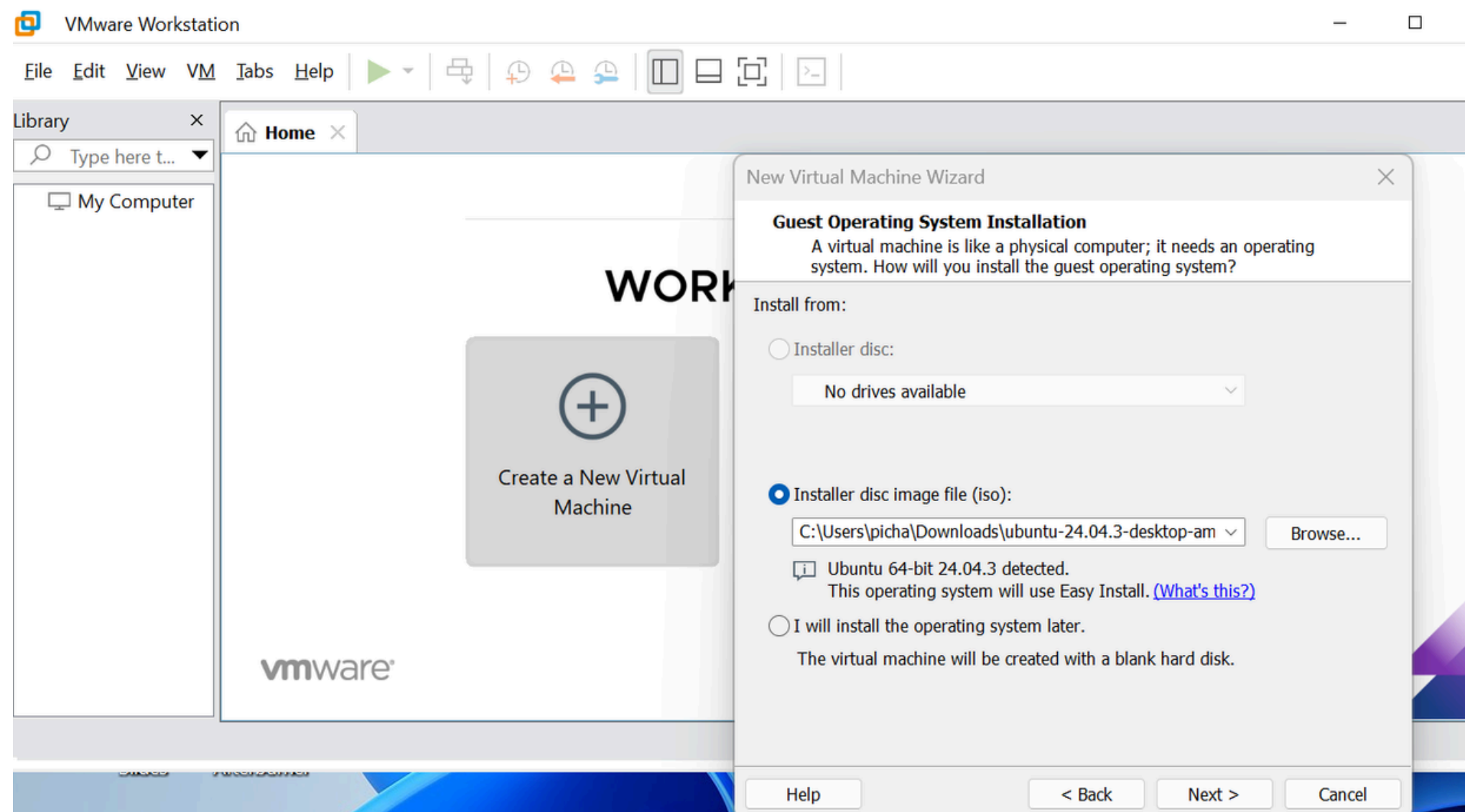
```
C:\shopdee> docker push username/nextjs:latest
```

3.6 การจำลอง Server

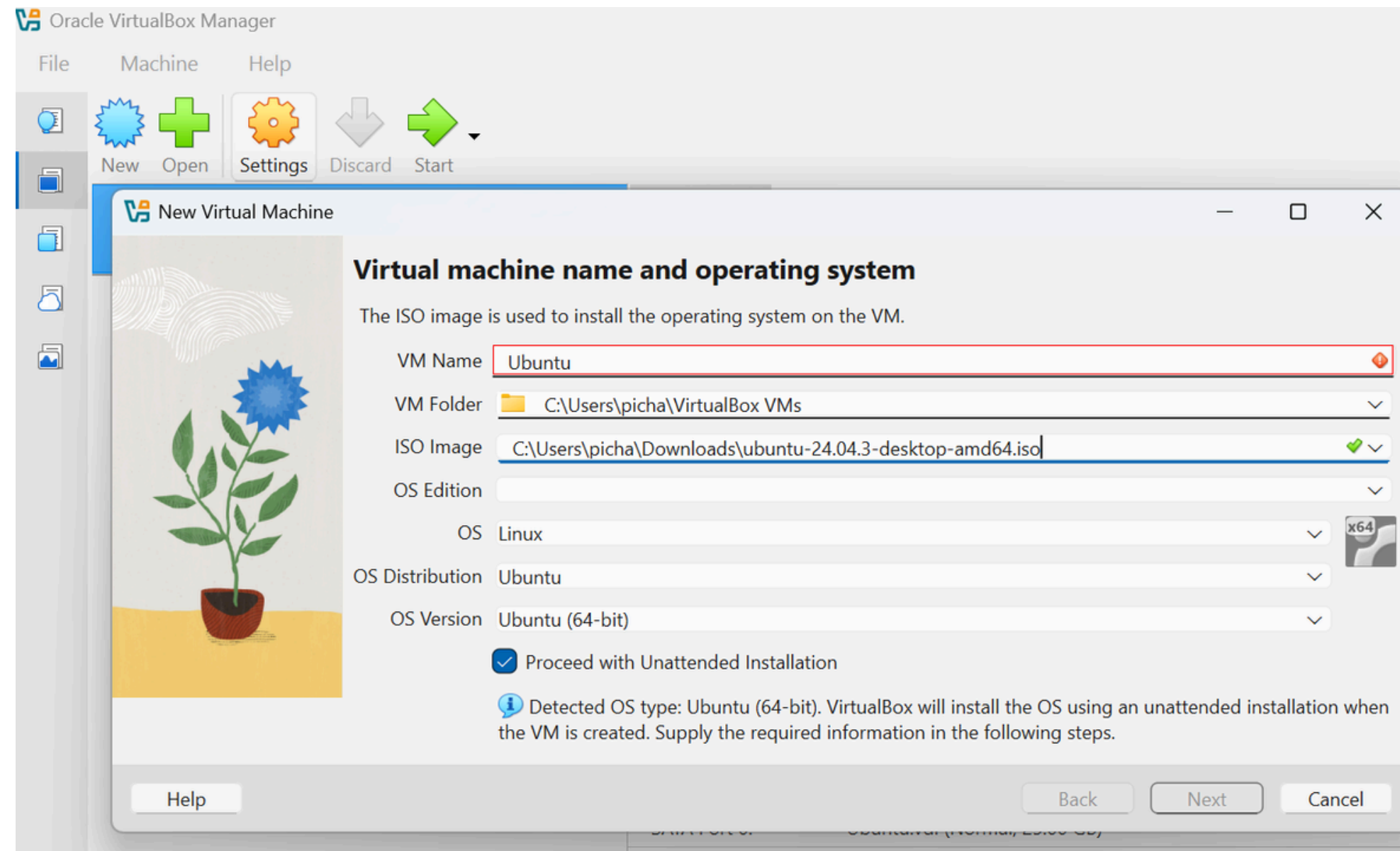
- ดาวน์โหลด Ubuntu ได้จาก <https://ubuntu.com/download/desktop>
 - ทำการติดตั้ง Ubuntu ลงบน Virtualization Software โดยเลือกใน Hyper-V, VMware, หรือ VirtualBox ตัวใดตัวหนึ่ง
- ติดตั้งและเปิด Hyper-V เพื่อลง Ubuntu



- ติดตั้งและเปิด VMware เพื่อลง Ubuntu
 - ลงทะเบียน <https://profile.broadcom.com/web/registration>
 - เข้าสู่ระบบ <https://support.broadcom.com/c/portal/login>
 - ดาวน์โหลด ได้จากเมนู “My Downloads” โดยคลิกที่ “Free Software Downloads available HERE” แล้วเลือก “VMware Workstation Pro” จากนั้นทำตามขั้นตอนที่เว็บแนะนำไปเรื่อย ๆ จนดาวน์โหลดสำเร็จ



- ติดตั้งและเปิด VirtualBox เพื่อลง Ubuntu
 - ดาวน์โหลด VirtualBox ได้จาก <https://www.virtualbox.org/wiki/Downloads>
 - ติดตั้ง Ubuntu โดยกำหนด VM Name และเลือก ISO Image จากไฟล์ที่ดาวน์โหลดมา



- กำหนด Network Adapter Types (Bridged)

ตารางเปรียบเทียบ Network Adapter Types

Adapter Types	การมองเห็นระหว่าง Host-Guest	การมองเห็นระหว่าง เครื่องอื่น-Guest	หมายเหตุ
Bridged	มองเห็นซึ่งกันและกัน	มองเห็นซึ่งกันและกัน	ใช้ IP วงเดียวกันกับ Host ออก Internet ได้
Host-only	มองเห็นซึ่งกันและกัน	มองไม่เห็นกันทั้งคู่	สร้าง IP วงใหม่เพื่อสื่อสารกัน ออก Internet ไม่ได้
NAT	Guest เห็น Host	Guest เห็นเครื่องอื่น	สร้าง NAT router เพื่อเป็นตัวกลางในการสื่อสาร ออก Internet และเครื่องอื่น ได้ Host และเครื่องอื่น เห็น Guest ผ่านการทำ Port Forwarding

หมายเหตุ : WSL จะมีลักษณะคล้ายกับ NAT+ Host-only

- การติดตั้ง docker engine

- สร้างไฟล์ install_docker.sh ดังนี้

```
#!/bin/bash
# Script ติดตั้ง Docker Engine uu Ubuntu

set -e

# 1. อัปเดตระบบ
sudo apt update
sudo apt upgrade -y

# 2. ติดตั้งแพ็คเกจที่จำเป็น
sudo apt install -y ca-certificates curl gnupg lsb-release

# 3. เพิ่ม GPG key ของ Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# 4. เพิ่ม repository ของ Docker
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 5. อัปเดตรายการแพ็คเกจ
sudo apt update

# 6. ติดตั้ง Docker Engine
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 7. เพิ่มสิทธิ์ให้รัน Docker โดยไม่ใช้ sudo (optional)
sudo usermod -aG docker $USER

# 8. แสดงเวอร์ชัน Docker
docker --version
echo "ติดตั้ง Docker เสร็จแล้ว! รีบูต session หรือใช้ 'newgrp docker' เพื่อใช้ docker โดยไม่ต้อง sudo."
```

- ให้สิทธิ์รับไฟล์

```
$ chmod +x install_docker.sh
```

- รับสคริปต์

```
$ ./install_docker.sh
```

เมื่อติดตั้งเสร็จจะพบคำว่า “ติดตั้ง Docker เสร็จแล้ว!”

- ตรวจสอบเวอร์ชันของ docker engine

```
$ docker --version
```

- ตรวจสอบสถานะการทำงานของ docker engine

```
$ sudo systemctl status docker
```

- ทำการ Start Docker (หาก Docker อยู่ในสถานะ **Inactive**)

```
$ sudo systemctl start docker
```

- ทำการ Stop Docker

```
$ sudo systemctl stop docker
```

- ทำการ Restart Docker

```
$ sudo systemctl restart docker
```

- ตั้งค่าให้ Docker เริ่มอัตโนมัติเมื่อเปิดเครื่อง

```
$ sudo systemctl enable docker
```

3.7 การ Pull Image และ Run Container บน Server

- คัดลอก docker-compose.yml, default.conf, และ .env มาไว้ในเครื่อง Ubuntu
- login ด้วย docker account

```
$ docker login -u YOUR-DOCKER-HUB-USER-NAME
```

ใช้ “personal access token” จาก Docker Hub แทน password

- ทำการ run docker compose

```
$ docker compose up -d
```

- ตรวจสอบรายการไฟล์ images

```
$ docker images
```

- ตรวจสอบรายการ containers

```
$ docker ps -a
```

- หยุด container

```
$ docker stop YOUR-IMAGE-ID-OR-NAME
```

- ลบ container

```
$ docker rm YOUR-IMAGE-ID-OR-NAME
```

- ลบไฟล์ image

```
$ docker rmi YOUR-CONTAINER-ID-OR-NAME
```

4. ทดสอบความมั่นคงปลอดภัย

- ติดตั้ง kali linux มีคำสั่ง ดังนี้ **Note** : หากมี ubuntu ที่ติดมากับ wsl อยู่แล้ว ไม่จำเป็นต้องติดตั้ง kali linux เพิ่ม แต่ยังคงติดตั้ง kali tool

```
C:\> wsl --install -d kali-linux
```

ในติดตั้งระบบจะให้ทำการกำหนด username และ password สำหรับ linux

- เข้าใช้งาน kali-linux

```
C:\> wsl -d kali-linux
```

หากติดตั้ง kali-linux เป็น default distro สามารถใช้คำสั่ง ดังนี้

```
C:\> wsl
```

หรือเข้าโดยค้นหาจาก “ช่องค้นหา” ของ Windows

- ติดตั้ง kali tool ที่ต้องใช้ เช่น nmap, hydra

```
$ sudo apt update  
$ sudo apt install nmap hydra
```

หากต้องการติดตั้งทั้ง 10 packages ให้ใช้คำสั่งนี้ `sudo apt install kali-tools-top10 -y`

- ติดตั้ง wordlists

```
$ sudo apt install wordlists
```

- ตรวจสอบไฟล์ password ที่ได้จาก wordlists

```
$ ls /usr/share/wordlists
```

จะพบไฟล์ 3 ไฟล์ดังนี้

1.john.lst เป็นไฟล์ password ขนาดเล็กมีประมาณ 3 พันกว่า passwords

2.nmap.lst เป็นไฟล์ password ขนาดเล็กมีประมาณ 5 พัน passwords

3.rockyou.txt.gz เป็นไฟล์ password ขนาดใหญ่มีประมาณ 14 ล้าน passwords ซึ่งถูก zip ไว้

หากต้องการใช้งานให้ unzip ก่อน โดยใช้คำสั่ง `sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`

- การใช้ nmap เพื่อตรวจสอบเครื่องเป้าหมายว่าเปิด Services ไหนบ้าง

```
$ nmap หมายเลข IP เครื่องเป้าหมาย หรือ เครื่อง Server
```

- ตัวอย่างการใช้ hydra เพื่อ Crack Password

```
hydra -l somchai@gmail.com -P /usr/share/wordlists/rockyou.txt 192.168.1.102  
http-post-form "/api/login:username=^USER^&password=^PASS^:F=ชื่อผู้ใช้หรือรหัส  
ผ่านไม่ถูกต้อง|Invalid username or password" -f -V
```

สามารถใช้ Developer Tools ของ Web Browser ในการตรวจหาค่าต่อไปนี้

- API end point : เช่น 192.168.1.102/api/login
- HTTP Request Method : เช่น post, get
- ตัวแปรรับ username ของ API : เช่น username, user_name, users
- ตัวแปรรับ password ของ API : เช่น password, pwd
- Return Message ของ API : เช่น ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง, Invalid username or password

THANK YOU

FOR YOUR ATTENTION



Contact Me:

 ห้อง 970

 pichai_jo@rmutto.ac.th

 busit.rmutto.ac.th

